

Esplorazione del database Chinook tramite un'interfaccia grafica e MongoDB¹

COURSE NAME

Big Data Analytics

Author:

Simone Sbreglia

31 maggio 2025

UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA



¹Repository codice: https://github.com/simonesbreglia/NoSQL_project

Indice

Abstract	2
1 MongoDB	2
1.1 Introduzione	2
1.2 Modalità di utilizzo	2
1.3 Operazioni CRUD	3
1.3.1 Insert	4
1.3.2 Delete	4
1.3.3 Update	4
1.3.4 Read	5
1.4 Aggregazione	5
2 Sperimentazione	6
2.1 Introduzione	6
2.2 Trasformazione del database	7
2.3 Interrogazione del database	8
2.3.1 Musica	8
2.3.2 Vendite	8
3 Interfaccia grafica	9
3.1 music.py	9
3.2 sales.py	9
4 Conclusioni	9
References	11
A Trasformazione del database	12
B Interrogazione del database: parte musicale	13

Abstract

In questo report si esporrà il funzionamento e le caratteristiche di MongoDB, un database NoSQL. In particolare verranno presentate le funzionalità di questo database, il modo in cui si possono memorizzare i dati e come effettuare le principali operazioni CRUD (Create, Read, Update, Delete). In questo specifico caso di utilizzo MongoDB è stato utilizzato per memorizzare i dati di un negozio musicale, riadattando la struttura nativa relazionale del database in una struttura NoSQL più adatta a MongoDB. Tutte le operazioni sono state effettuate tramite python, utilizzando la libreria pymongo. Infine è stata sviluppata una interfaccia grafica in streamlit per interrogare in modo dinamico il database, sia per la parte musicale che per la parte amministrativa e di vendite. Il codice sorgente è disponibile al repository GitHub disponibile al link fornito.

1 MongoDB

1.1 Introduzione

MongoDB è un database NoSQL, ossia un database non relazionale che archivia i dati in modo flessibile. MongoDB offre un servizio su Cloud, MongoDB Atlas e una versione MongoDB Community, installabile sulla propria macchina in locale. Per gli scopi di questo progetto è stato installato MongoDB Community in locale. MongoDB è pensato per essere scalato orizzontalmente, distribuendo i dati su più macchine e con varie repliche, per garantire alta disponibilità e tolleranza ai guasti.

Un record in MongoDB è un documento, ossia una struttura dati composta da coppie di chiavi a valori; questi valori potrebbero essere a loro volta documenti, oppure vettori o valori. La struttura di un documento è simile a quella di un oggetto JSON, ossia un oggetto JavaScript che rappresenta una struttura dati composta da coppie di chiavi a valori. In particolare MongoDB salva i documenti in formato BSON, ossia una rappresentazione binaria dei file JSON. Un esempio di documento è il seguente:

Listing 1: Esempio di JSON

```
1  {
2      "Nome": "Paolo",
3      "Professione": "Studente",
4      "TitoliDiStudio": ["Diploma", "Laurea Triennale"],
5      "Indirizzo": {
6          "Via": "Campi"
7          "NumeroCivico": "213/b",
8          "Cap": 41125,
9          "Citta": "Modena"
10     },
11     "Anni": 24,
12 }
```

L'utilizzo di documenti come record per il salvataggio di dati ha diversi vantaggi, infatti documenti di questo tipo corrispondono al tipo nativo di dati in molti linguaggi di programmazione. Inoltre la possibilità di annidare documenti e di inserire array come valori permette di avere strutture di dati che diminuiscono le operazioni di join. MongoDB memorizza i documenti in *collezioni*, ossia l'equivalente delle tabelle in database relazionali. Inoltre MongoDB supporta anche:

- **Viste di sola lettura:** viste virtuali basate su una query che mostrano una finestra sui dati di una collezione esistente, aggiornata automaticamente ogni volta che viene consultata la finestra
- **Viste materializzate su richiesta:** una tabella o una collezione che memorizza i risultati di una query per migliorare le prestazioni, non vengono aggiornate automaticamente ma solo manualmente o tramite script

1.2 Modalità di utilizzo

MongoDB può essere utilizzato in diversi modi, tra cui:

- MongoDB Shell
- MongoDB Compass
- MongoDB Atlas

- Driver MongoDB

MongoDB Shell è un'interfaccia a riga di comando che permette di interagire con il database, eseguire query e comandi di amministrazione. La shell risulta utile per eseguire operazioni rapide e testare comandi, ma non è adatta all'uso in produzione.

MongoDB Compass è un'interfaccia grafica che permette di esplorare i dati, eseguire query e visualizzare i risultati, fornisce uno strumento utile per analizzare i dati e per eseguire operazioni di amministrazione, ma non è adatta per l'uso in produzione.

MongoDB Atlas è un servizio di database as a service (DBaaS) che permette di creare e gestire cluster MongoDB in cloud. Atlas offre funzionalità avanzate come il backup automatico, la scalabilità automatica e la sicurezza integrata previo pagamento, ma mette a disposizione anche un piano gratuito con una capacità massima di 512 MB e alcune limitazioni per backup e replica.

Driver MongoDB sono librerie che permettono di interagire con MongoDB da diversi linguaggi di programmazione, come Python, Java, Node.js e altri. I driver offrono un'interfaccia per eseguire operazioni CRUD e per gestire le connessioni al database. Per questo progetto è stato utilizzato il driver **pymongo** per Python. Utilizzare un driver significa potersi appoggiare a un linguaggio di programmazione per costruire query ed effettuare operazioni in modo dinamico e automatico.

1.3 Operazioni CRUD

Le operazioni CRUD (Create, Read, Update, Delete) sono le operazioni fondamentali per interagire con un database. In MongoDB queste operazioni vengono eseguite tramite metodi specifici. Di seguito sono riportati alcuni esempi di operazioni CRUD utilizzando MongoDB Shell. Supponiamo di avere in locale MongoDB Compass e MongoDB Shell installati, allora le istruzioni a seguire permettono di creare un database *Scuola* e due collezioni, *Studenti* e *Professori*.

Listing 2: Esempio di operazioni CRUD in MongoDB Shell

```
1  use Scuola
2  db.createCollection(
3      "Studenti", {
4      validator: {
5          $jsonSchema: {
6              bsonType: "object",
7              title: "Validazione dell'oggetto Studente",
8              required: ["Id", "Nome", "Cognome"],
9              properties: {
10                  Id: { bsonType: "int" },
11                  Nome: { bsonType: "string" },
12                  Cognome: { bsonType: "string" },
13                  DataDiNascita: { bsonType: "date" }
14              }
15          }
16      }
17  )
18  db.Studenti.createIndex({ "Id": 1 }, { unique: true })
19  db.createCollection(
20      "Professori", {
21      validator: {
22          $jsonSchema: {
23              bsonType: "object",
24              title: "Validazione dell'oggetto Studente",
25              required: ["Id", "Nome", "Cognome"],
26              properties: {
27                  Id: { bsonType: "int" },
28                  Nome: { bsonType: "string" },
29                  Cognome: { bsonType: "string" },
30                  DataDiNascita: { bsonType: "date" }
31              }
32          }
33      }
34  )
35  db.Professori.createIndex({ "Id": 1 }, { unique: true })
```

Le operazioni CRUD mostrate creano un database Scuola, due collezioni, Studenti e Professori, e definiscono uno schema per la validazione dei documenti: è richiesto che siano inseriti almeno un Id, un Nome e un Cognome. Inoltre viene richiesto, tramite la creazione un indice, che gli Id siano univoci nelle rispettive collezioni. Se durante una fase di insert non vengono soddisfatti questi requisiti, MongoDB non permette di inserire il documento e restituisce un errore.

1.3.1 Insert

MongoDB permette di inserire documenti tramite dei metodi, *insert_one* e *insert_many*, che nello specifico inseriscono uno o più documenti in una collezione. Di seguito sono riportati alcuni esempi di inserimento di documenti.

Listing 3: Esempio di insert in MongoDB

```
1 db.Studenti.insertOne({
2     Id: 2,
3     Nome: "Luca",
4     Cognome: "Rossi",
5     DataDiNascita: ISODate("2000-05-15T00:00:00Z")
6 })
7 db.Studenti.insertMany([
8     {
9         Id: 2,
10        Nome: "Stefano",
11        Cognome: "Verdi",
12        DataDiNascita: ISODate("2000-05-15T00:00:00Z")
13    },
14    {
15        Id: 3,
16        Nome: "Anna",
17        Cognome: "Bianchi",
18        DataDiNascita: ISODate("1999-11-20T00:00:00Z")
19    },
20    {
21        Id: 4,
22        Nome: "Marco",
23        Cognome: "Galli",
24        DataDiNascita: ISODate("2001-01-10T00:00:00Z")
25    }
26 ])
```

In modo analogo si possono inserire dei documenti nella collezione Professori.

1.3.2 Delete

MongoDB permette di eliminare documenti tramite i metodi *delete_one* e *delete_many*, che eliminano uno o più documenti in una collezione. Di seguito sono riportati alcuni esempi di eliminazione di documenti.

Listing 4: Esempio di delete in MongoDB

```
1 db.Studenti.deleteOne({ Id: 2 })
2 db.Studenti.deleteMany({ Nome: "Marco" })
3 db.Studenti.deleteMany({ Nome: { $in: ["Marco", "Luca"] } })
4 db.Studenti.deleteMany({ Nome: { $nin: ["Marco", "Luca"] } })
```

Il primo comando elimina il documento con Id 2, il secondo elimina tutti i documenti in cui il campo "Nome" è "Marco", il terzo elimina tutti i documenti in cui il campo "Nome" è "Marco" o "Luca", mentre il quarto elimina tutti i documenti in cui il campo "Nome" non è "Marco" o "Luca".

1.3.3 Update

MongoDB permette di aggiornare documenti tramite i metodi *update_one* e *update_many*, che aggiornano uno o più documenti in una collezione. Durante gli update vengono spesso usate pipeline di aggregazione, che permettono di concatenare più azioni una dopo l'altra. In seguito verranno approfondite le pipeline di aggregazione, per il momento sono riportati alcuni esempi di aggiornamento di documenti.

Listing 5: Esempio di update in MongoDB

```

1  db.Studenti.updateOne(
2      { Id: 2 },
3      { $set: { Nome: "Marco" } }
4  )
5  db.Studenti.updateMany(
6      { Nome: "Marco" },
7      { $set: { Nome: "Luca" } }
8  )
9  db.Studenti.updateMany(
10     { Corso: { $exists: false } },
11     { $set: { Corso: "Matematica" } }
12 )

```

Il primo comando aggiorna il documento con Id 2, cambiando il campo Nome in Marco, il secondo cambia il campo Nome in Luca per tutti i documenti in cui il campo Nome è Marco, mentre il terzo aggiorna tutti i documenti in cui il campo Corso non esiste, aggiungendo il campo Corso con valore Matematica.

1.3.4 Read

MongoDB permette di leggere documenti tramite i metodi *findOne*, *find* che restituiscono uno o più documenti in una collezione. Entrambi i metodi prendono in input un filtro che permette di specificare quali documenti si vogliono leggere, espressi in formato JSON, e un parametro di proiezione che permette di specificare quali campi di vogliono restituire e con quale nome, sempre in formato JSON. Di seguito sono riportati alcuni esempi di lettura di documenti.

Listing 6: Esempio di read in MongoDB

```

1  db.Studenti.find()
2  db.Studenti.find({ Id: 2 })
3  db.Studenti.find({ Corso: "Matematica" })
4  db.Studenti.find(
5      { Corso: "Matematica" },
6      { Nome: 1, Cognome: 1, _id: 0 }
7  )

```

Il primo comando restituisce tutti i documenti nella collezione Studenti, il secondo restituisce il documento con Id 2, il terzo restituisce tutti i documenti in cui il campo Corso è Matematica, mentre il quarto restituisce solo i campi Nome e Cognome dei documenti in cui il campo Corso è Matematica, escludendo il campo `_id`.

1.4 Aggregazione

MongoDB mette a disposizione un potente framework di aggregazione che permette di eseguire operazioni complesse sui dati, come filtri, ordinamenti, raggruppamenti, join, somme e molto altro. Le operazioni di aggregazione non sono altro che delle pipeline che contengono un operatore dopo l'altro, ognuno dei quali esegue un'operazione sui dati e restituisce il risultato al successivo operatore. In Tabella 1, sono riportati i principali operatori di aggregazione disponibili in MongoDB. Per visualizzare la lista completa degli operatori di aggregazione è possibile consultare la documentazione ufficiale di MongoDB, vedi [2].

Operatore	Descrizione
\$match	Filtra i documenti secondo una condizione, simile a una WHERE in SQL.
\$group	Raggruppa i documenti in base a uno o più campi e calcola valori aggregati.
\$sort	Ordina i documenti secondo uno o più campi, in ordine crescente o decrescente.
\$project	Seleziona e ristruttura i campi del risultato, può anche creare campi nuovi.
\$limit	Limita il numero di documenti nel risultato finale.
\$skip	Salta un certo numero di documenti dall'inizio del risultato.
\$count	Conta il numero di documenti e restituisce il totale in un campo chiamato count.
\$lookup	Effettua una "join" tra collezioni, simile a una JOIN in SQL.
\$unwind	Divide un array in più documenti singoli, uno per ogni elemento dell'array.
\$addFields	Aggiunge nuovi campi o modifica quelli esistenti nei documenti.
\$replaceRoot	Sostituisce il documento intero con un altro oggetto specificato.
\$merge	Scriva il risultato dell'aggregazione in una nuova collezione o aggiorna una esistente.

Tabella 1: Operatori di aggregazione più comuni in MongoDB

La concatenazione di pipeline di aggregazione permette di eseguire operazioni complesse direttamente sul database, sfruttando tutte le ottimizzazioni e le potenzialità di MongoDB. Per alcuni esempi, vedere la Sezione [2](#).

2 Sperimentazione

2.1 Introduzione

Lo scopo del progetto è quello di sviluppare competenze legate a MongoDB, capire come funziona, come gestire i dati e come sfruttare il database da altri linguaggi di programmazione, come ad esempio Python.

A tale scopo è stato scelto un dataset, chiamato Chinook Database [1]. Il database originale è un database relazionale, composto da 11 tabelle, che raccolgono informazioni su tracce musicali, artisti, album e alcune playlist, insieme alle informazioni sulle vendite, sui clienti e sugli impiegati del negozio musicale. In Figura 1 è riportato lo schema del database originale, composto da 11 tabelle. Il database è stato convertito in un formato NoSQL, creando quattro collezioni:

- *Customer*: contiene le informazioni sui clienti
- *Employee*: contiene le informazioni sugli impiegati
- *Invoice*: contiene le informazioni sulle fatture
- *Track*: contiene le informazioni sulle tracce musicali

Trasformando il database seguendo questa struttura si sfruttano le potenzialità di MongoDB, riducendo il numero di join e aumentando le prestazioni in fase di lettura dei dati. In Figura 2 è riportato lo schema del database NoSQL.

Il repository indicato contiene il codice sorgente del progetto e i dati utilizzati durante la sperimentazione. La struttura del repository e le istruzioni per l'installazione sono riportate nel file *README.md*. Nel resto della sezione verranno approfonditi alcuni aspetti del progetto, in particolare la trasformazione del database e l'interrogazione del database.

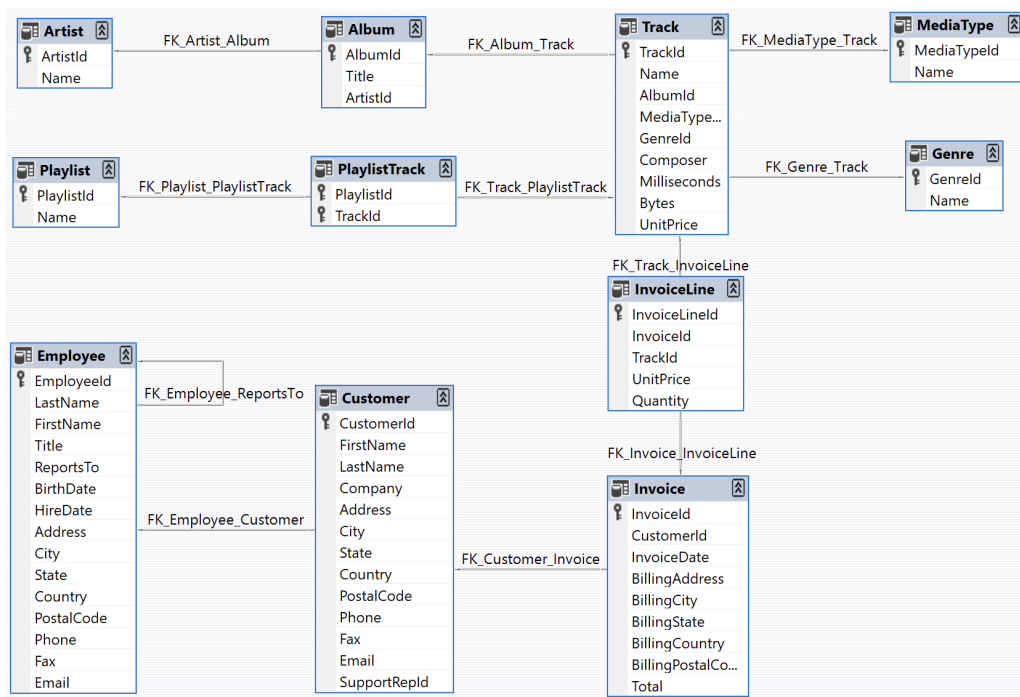


Figura 1: Schema del database Chinook

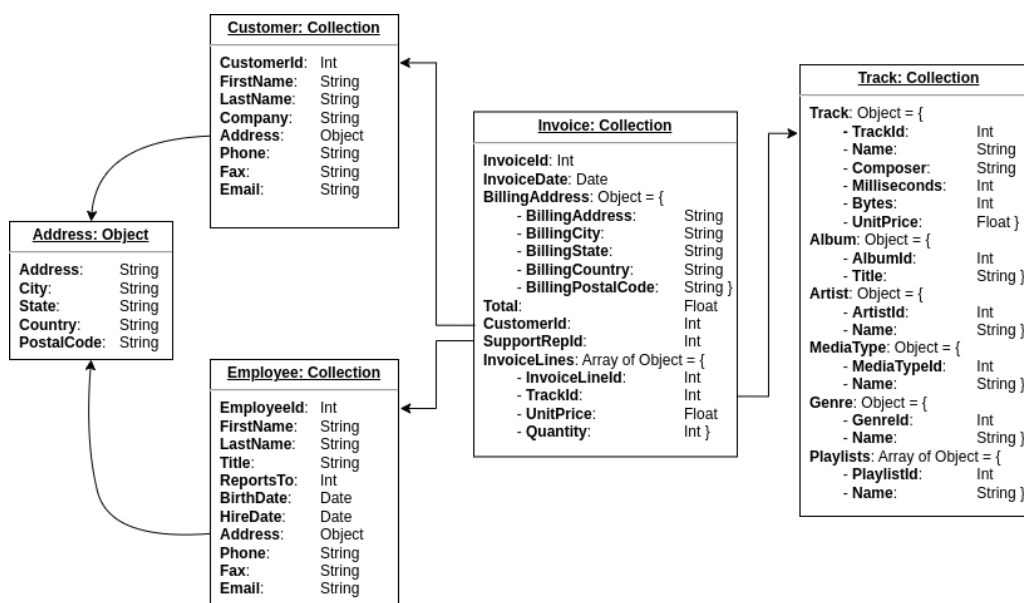


Figura 2: Schema del database Chinook NoSQL

2.2 Trasformazione del database

Come introdotto nella sottosezione precedente, il database originale è stato trasformato in un formato adatto a un database NoSQL, puntando a ridurre il numero di join e a sfruttare la potenza e la flessibilità di MongoDB. Per ridurre il numero di join durante le query, le informazioni relative alla musica sono state incorporate in un unico documento per ogni traccia presente nel database. Questo approccio genera rindondanza, nel senso che ogni alub, ogni artista e ogni genere viene salvato più volte, una pper ogni traccia, ma permette di ridurre il numero di operazioni con l'operatoe \$lookup e di velocizzare le operazioni di lettura, essendo questo il più costoso.

Per quanto riguarda le informazioni relative a clienti, impiegati e fatture, sono state create tre collezioni separate. La motivazione è che ogni fattura si collega a un cliente, un impiegato e un certo numero di tracce musicali in base all'acquisto. Creare una collezione unica per le fatture avrebbe implicato la rindondanza delle informazioni relative a clienti e impiegati, contenenti una discreta quantità di dati. Questo approccio permette

quindi di risparmiare spazio e effettuare un piccolo numero di join, non ottimale ma le prestazioni non ne risentono nella maggior parte dei casi.

Per inserire i dati nel database sono stati utilizzati due script python, uno che fornisce delle funzioni per la connessione a MongoDB in vari casi: locale, remoto o in Docker, e uno che si occupa di trasformare i dati da formato CSV a formato JSON, per poi inserirli nel database. Il codice sorgente per la connessione a MongoDB si trova nel file *db_utils.py* mentre il codice per la trasformazione del database si trova nel file *transform_db.py*.

2.3 Interrogazione del database

L'interrogazione del database si divide in due parti: la parte musicale e la parte dedicata alle vendite.

La sezione musicale permette di interrogare il database per ottenere informazioni su tracce, album e artisti. La sezione vendite invece permette di interrogare il database per ottenere informazioni su fatture, clienti, impiegati e statistiche di vendita.

Nel seguito sono riportati alcuni esempi di query per entrambe le sezioni.

2.3.1 Musica

La pagina dedicata alla musica permette di interrogare il database per ottenere informazioni su tracce, album e artisti. In particolare offre la possibilità di effettuare tre tipi di query: contare, raggruppare e contare, filtrare.

Per quanto riguarda le query di conteggio, è possibile contare il numero di tracce, album, artisti, playlist e generi.

Per le query di raggruppamento invece è possibile contare:

- numero di tracce per album
- numero di tracce per genere
- numero di tracce per artista
- numero di tracce per playlist
- numero di artisti per playlist
- numero di album per artista

Nei listati 9 e 10 sono riportati due esempi di query di conteggio e di raggruppamento.

La parte dedicata al filtraggio invece è più complessa e permette di creare filtri personalizzati per ricercare tracce, album, artisti e qualsiasi dato presente nel database. L'esempio mostrato in Figura ?? mostra ad esempio una serie di filtri che vanno a cercare tutte le tracce che hanno come artista "AC/DC" e che hanno una lunghezza compresa tra i 3 minuti e i 4 minuti. Inoltre è possibile selezionare le informazioni da visualizzare, come ad esempio il costo della traccia, il genere, il nome dell'album, ecc. Questo strumento permette di creare query di ricerca personalizzate, concatenando i vari filtri.

Il codice sorgente per il filtraggio della musica in modo dinamico comprende una parte che si occupa di creare i filtri, gestita completamente da Streamlit, e una parte che si occupa di eseguire la query sul database, costruendola sulla base dei filtri aggiunti. Nel listato 11 è riportato il codice sorgente per la costruzione della query.

2.3.2 Vendite

La pagina dedicata alle vendite permette di ottenere informazioni su fatture, clienti, impiegati e statistiche di vendita. In particolare offre la possibilità di visualizzare:

- vendite per cliente totali e distribuite nel tempo
- vendite per impiegato totali e distribuit nel tempo
- vendite in base alla musica divisa per traccia, album, artista o genere

Questo tipo di query richiede più operazioni rispetto alle query della sezione precedente, in quanto è richiesto di effettuare dei join, dei filtri in base a cosa si seleziona, dei raggruppamenti in base alla data di vendita, al cliente, all'impiegato o infine in base al filtro musicale selezionato. Per ogni tipo di query sono state scritte delle pipeline, salvate in formato JSON, a cui vanno aggiunti in modo dinamico gli Id delle tracce, degli album, degli artisti, dei generi, dei clienti e degli impiegati, in base al tipo di query selezionato. Le query sono raccolte nella cartella PipelinesMongoDB. Per un esempio di query vedere il listato 12.

3 Interfaccia grafica

Per rendere fruibile il progetto e per interrogare il database in modo dinamico è stata sviluppata un'interfaccia grafica utilizzando la libreria Python Streamlit. L'interfaccia è composta da tre pagine:

- *app.py*: pagina principale, reindirizza alle altre due pagine in base alla scelta dell'utente
- *music.py*: pagina dedicata alla musica, permette di interrogare il database per ottenere informazioni su tracce, album e artisti
- *sales.py*: pagina dedicata alle vendite, permette di interrogare il database per ottenere informazioni su fatture, clienti, impiegati e statistiche di vendite

Nel seguito vengono descritte le due pagine principali, *music.py* e *sales.py*.

3.1 music.py

La pagina dedicata alla musica offre una prima scelta tramite un menù a tendina, che permette di selezionare il tipo di query da eseguire. Le query disponibili sono di conteggio, di raggruppamento o di filtraggio. Per le prime due opzioni viene poi presentato un ulteriore menù a tendina che permette di selezionare il tipo di conteggio o di raggruppamento da eseguire. Se invece viene selezionato il filtraggio, viene aperta una sezione che permette di aggiungere filtri a piacimento per la musica. Comparirà un tasto *Add filter* che permette di aggiungere un filtro usando le finestre che compariranno. Ogni filtro può essere rimosso in qualsiasi momento. Cliccando sul tasto *Execute query* verrà eseguita la query e verranno mostrati i risultati in una tabella, dove le colonne mostrate sono quelle selezionate in fase di filtraggio.

Questo strumento, nella sua semplicità, concatena un filtro dopo l'altro, permettendo all'utente di trovare tutta la musica disponibile sul catalogo che soddisfa determinati parametri. Inoltre, la possibilità di selezionare le colonne da visualizzare permette all'utente di non vedere informazioni non necessarie, rendendo la tabella più leggibile e comprensibile.

3.2 sales.py

In questa sezione viene presentato un primo menù a tendina che permette di selezionare il tipo di query da eseguire. Le opzioni sono:

- *Sales by Employee*
- *Sales by Customer*
- *Sales by Music*

Sales by Employee permette di selezionare uno o più ruoli di impiegati e poi uno o più impiegati in base ai ruoli scelti. Schiacciando poi sul tasto *Execute query* verrà eseguita la query e verranno mostrati due grafici: uno per le vendite totali per impiegato, e uno per le vendite distribuite nel tempo per impiegato.

Sales by Customer permette di selezionare uno o più clienti e disegna due grafici: uno per gli acquisti totali per cliente e uno per gli acquisti distribuiti nel tempo per cliente.

Sales by Music permette di selezionare un'opzione tra traccia, album, artista o genere. In base alla selezione compariranno le tracce, gli album, gli artisti o i generi disponibili. L'utente può selezionare uno o più elementi e poi schiacciare sul tasto *Execute query*. Verranno mostrati due grafici: uno per le vendite totali per elemento selezionato e uno per le vendite distribuite nel tempo per elemento selezionato. In questo modo, ad esempio, è possibile comparare le vendite dei vari generi musicali.

Tutti i grafici sono interattivi e permettono di escludere informazioni, ingrandire o rimpicciolire i grafici, e di visualizzare le informazioni in modo più chiaro.

Inoltre, tutti i menù a tendina che compaiono sfruttano delle query eseguite su MongoDB, in modo da ottenere gli Id degli elementi selezionati. Le funzionalità di MongoDB non vengono sfruttate solo per le query, ma anche per la costruzione di questi menù a tendina dinamici.

4 Conclusioni

In questo report sono stati presentati i principali aspetti di MongoDB, tuttavia la funzionalità di questo database sono molteplici e non tutte possono essere sfruttate con il database Chinook. Durante la sperimentazione ho personalmente avuto modo di approfondire le funzionalità di MongoDB, in particolare le operazioni CRUD e le

pipeline di aggregazione, che rappresentano uno strumento indispensabile per interrogare in modo efficiente il database. Inoltre ho avuto modo di imparare a utilizzare i driver MongoDB, in particolare il driver pymongo, che permette di utilizzare MongoDB in modo semplice, veloce e dinamico. La possibilità di implementare script in Python per manipolare, interrogare e visualizzare i dati offre un'ottima opportunità per approfondire le proprie conoscenze per entrambi gli strumenti.

Per quanto riguarda le prestazioni di MongoDB, alcune query eseguite sono piuttosto pesanti per un database NoSQL, in quanto richiedono un numero di join elevato e una serie di filtri e raggruppamenti. Tuttavia, MongoDB riesce a gestire in modo efficiente anche questo tipo di query. La flessibilità di poter salvare i dati in documenti JSON, unita alla capacità di eseguire operazioni complesse tramite pipeline di aggregazione, rende MongoDB uno strumento potente e versatile per la gestione dei dati. Inoltre, la possibilità di scalare orizzontalmente il database e di distribuire i dati su più macchine permette di gestire grandi volumi di dati in modo efficiente.

A progetto concluso ho avuto l'opportunità di utilizzare MongoDB al di fuori del contesto universitario, in un progetto aziendale in cui mi sono ritrovato a dover gestire lo storage di un grande volume di documenti, in particolare i dati venivano registrati con una frequenza media di 4ms. MongoDB si è rivelato uno strumento utile, in grado di gestire questa frequenza di inserimento di dati, senza rallentare il flusso generale del progetto in questione.

Riferimenti bibliografici

[1] Kaggle (2025). Chinookdataset. Accessed: 2025-04-21.

[2] MongoDB, Inc. (2025). Mongoddb documentation. Accessed: 2025-04-21.

A Trasformazione del database

Il codice nel Listato 7 mostra le funzioni per la connessione a MongoDB, in ambiente Docker, in locale o su MongoDB Atlas.

Listing 7: Connessione a MongoDB

```
from pymongo.mongo_client import MongoClient
from pymongo.server_api import ServerApi

def get_docker_connection():
    url = "mongodb://mongodb:27017/"
    client = MongoClient(url)
    # controlla se esiste il db Chinook
    db = client['Chinook']
    # elimina il db Chinook se esiste
    if db in client.list_database_names():
        client.drop_database(db)
    return db["Chinook"]

def get_remote_connection():
    uri = st.secrets['mongo']['uri']
    client = MongoClient(uri, server_api=ServerApi('1'))
    return client['Chinook']

def get_local_connection():
    url = "mongodb://localhost:27017/"
    client = MongoClient(url)
    return client['Chinook']

def ping_mongo(client):
    try:
        client.admin.command('ping')
    except Exception as e:
        return False
    return True
```

Listing 8: Inserimento della collezione Track

```
if os.environ.get('DOCKER_BUILD') == 'True':
    db = get_docker_connection()
else:
    db = get_local_connection()

Track = db['Track']
Invoice = db['Invoice']
Customer = db['Customer']
Employee = db['Employee']

dfs = {}
table_names = ['Album', 'Artist', 'Customer', 'Employee', 'Genre',
               'Invoice', 'InvoiceLine', 'MediaType',
               'Playlist', 'PlaylistTrack', 'Track']
path = 'data/ChinookDataset/'
for table in table_names:
    dfs[table] = pd.read_csv(path + table + '.csv')
    dfs[table].index = np.arange(1, len(dfs[table]) + 1)

for i in range(len(dfs['Track'])):
    record = dfs['Track'].loc[i+1]
    entry = {
        "Track": {
```

```

        "TrackId": int(record['TrackId']),
        "Name": record['Name'],
        "Composer": record['Composer'],
        "Milliseconds": int(record['Milliseconds']),
        "Bytes": int(record['Bytes']),
        "UnitPrice": float(record['UnitPrice'])
    },
    "Album": {
        "AlbumId": int(record['AlbumId']),
        "Title": dfs['Album'].loc[record['AlbumId']]['Title']
    },
    "Artist": {
        "ArtistId": int(dfs['Album'].loc[record['AlbumId']]['ArtistId']),
        "Name": dfs['Artist'].loc[dfs['Album'].loc[
            record['AlbumId']]['ArtistId']]['Name']
    },
    "MediaType": {
        "MediaTypeId": int(record['MediaTypeId']),
        "Name": dfs['MediaType'].loc[record['MediaTypeId']]['Name']
    },
    "Genre": {
        "GenreId": int(record['GenreId']),
        "Name": dfs['Genre'].loc[record['GenreId']]['Name']
    },
    "Playlist": [
        {
            "PlaylistId": int(actual_record['PlaylistId']),
            "Name": dfs['Playlist'].loc[actual_record['PlaylistId']]['Name']
        }
        for actual_record in dfs['PlaylistTrack'].loc[
            dfs['PlaylistTrack']['TrackId'] == (
                record['TrackId'] + 1)].to_dict(orient='records')
    ]
}

Track.insert_one(entry)

```

B Interrogazione del database: parte musicale

Nel listato 9 è mostrato il codice di una funzione che permette di contare il numero di tracce distinte nel database, riportando solo il nome della traccia. La funzione restituisce i nomi distinti delle tracce e una lista che contiene i nomi delle colonne da visualizzare nell'interfaccia grafica.

Listing 9: Esempio di query di conteggio

```

def count_tracks():
    return db.Track.distinct("Track.Name"), ["Number_of_Tracks", "Track_Title"]

```

Nel listato 10 è mostrato il codice di una funzione che permette di raggruppare le tracce in base all'album e di contarle. La funzione restituisce solamente il risultato della query, mentre i nomi delle colonne da visualizzare sono impliciti grazie alla proiezione. Il resto delle query di raggruppamento sono simili.

Listing 10: Esempio di query di raggruppamento

```

def count_songs_per_album():
    return list(db.Track.aggregate([
        {"$group": {"_id": "$Album.Title", "count": {"$sum": 1}}},
        {"$project": {"_id": 0, "Album": "$_id", "Number_of_tracks": "$count"}},
        {"$sort": {"Number_of_tracks": -1}}
    ]), None

```

Nel listato 11 è mostrato il codice di una funzione che permette di filtrare le tracce in base ai filtri selezionati dall'utente. Per il codice completo si rimanda al file *music.py* presente nel repository.

Listing 11: Esempio di query di filtraggio

```

if st.button("Execute_query"):
    and_filters = []
    project = {}
    for f in st.session_state.filters:
        if f['field'] is not None and
           f['operator'] is not None and
           f['value'] is not None:
            query = {fields[f["field"]]: {operator_map[f["operator"]]: f["value"]}}
            and_filters.append(query)

    project = {
        "_id": 0,
        "Track_Name": "$Track.Name" if "Track_Name" in sel_col else None,
        "Album_Title": "$Album.Title" if "Album_Title" in sel_col else None,
        "Genre": "$Genre.Name" if "Genre" in selected_col else None,
        "Artist_Name": "$Artist.Name" if "Artist_Name" in sel_col else None,
        "Composer": "$Track.Composer" if "Composer" in sel_col else None,
        "Playlist_Name": "$Playlist.Name" if "Playlist_Name" in sel_col else None,
        "Track_length": "$Track.Milliseconds" if "Track_length" in sel_col else None,
        "Price": "$Track.UnitPrice" if "Price" in sel_col else None,
        "Media_type": "$MediaType.Name" if "Media_type" in sel_col else None
    }

    # elimina le chiavi con valore None
    project = {k: v for k, v in project.items() if v is not None}

    query = {"$and": and_filters} if and_filters else {}
    # Esegui la query
    query_result = list(db.Track.find(query, project))

    # trasforma i millisecondi in minuti e secondi
    for q in query_result:
        if "Track_length" in q:
            q["Track_length"] = f"{q['Track_length']_//_60000}:"
                f"{{(q['Track_length']_%_60000)_//_1000:02}}"

    # Visualizza il risultato
    st.write(f"Number_of_results:_{len(query_result)}")
    st.dataframe(query_result)

```

Listing 12: Esempio di pipeline di aggregazione

```

1 [
2   {
3     "$match": {
4       "InvoiceLines.TrackId": { "$in": [] }
5     }
6   },
7   {
8     "$unwind": "$InvoiceLines"
9   },
10  {
11    "$lookup": {
12      "from": "Track",
13      "let": { "trackId": "$InvoiceLines.TrackId" },
14      "pipeline": [
15        {
16          "$match": {
17            "$expr": {
18              "$eq": [ "$Track.TrackId", "$$trackId" ]
19            }
20          }

```

```

21     }
22     ],
23     "as": "TrackVec"
24 }
25 },
26 { "$unwind": "$TrackVec" },
27 {
28     "$project": {
29         "_id": 0,
30         "TrackName": "$TrackVec.Track.Name",
31         "Date": "$InvoiceDate",
32         "TotalRevenue": {
33             "$multiply": [ "$InvoiceLines.UnitPrice", "$InvoiceLines.Quantity" ]
34         }
35     }
36 },
37 {
38     "$group": {
39         "_id": {
40             "TrackName": "$TrackName",
41             "Date": "$Date"
42         },
43         "TotalRevenue": { "$sum": "$TotalRevenue" }
44     }
45 },
46 {
47     "$project": {
48         "_id": 0,
49         "TrackName": "$_id.TrackName",
50         "Date": "$_id.Date",
51         "TotalRevenue": "$TotalRevenue"
52     }
53 },
54 { "$sort": { "_id.Date": 1, "TotalRevenue": -1 } }
55 ]

```